Shared readouts, not S^3: what drives forgetting in a toy quaternion network

Oct 25, 2026 · @Billy Moore

In this toy quaternion network, forgetting comes mostly from how the two tasks read the output, and the rest from writing each weight as amplitude × unit quaternion, not from the unit sphere itself. Most of this matches known results in ordinary networks. The forgetting cost of the length–direction split is the one piece we found unreported.

- **Parameterization, not geometry (the new piece).** The 9-point gap between S^3 and free weights is entirely the $A \cdot q$ factorization shrinking direction updates by A^2. Dropping the unit-norm constraint changes nothing (+0.0); undoing the A^2 scaling recovers +9.4 points [6.0, 13.0]. The cost survives Adam (+9.4 to +10.6, paired).
- **Readout (a known pattern).** Giving each task its own output quaternion recovers 15–23 points of Task A, measured when Task B first reaches 95%. Separate output heads are known to forget far less; here the i and j parts of one output quaternion act as a single shared head.
- **No quaternion advantage.** With both fixes, the quaternion network matches real MLPs of equal size or equal width to within 0.3 points.
- **Misread, not erased (a replication).** After Task B, a linear probe still reads Task A at 98.7–99.4% from every network's hidden layer. Refitting just the readout on 100 stored points recovers 90–96%.
- **Protection erases the differences.** With replay or refreshed OGD, every network keeps 95–97% of Task A; output anchoring does on five of six.

Setup

A small network learns Task A, then Task B, and we measure how much of Task A survives. Both tasks are binary classification on $(x_1, x_2) \in [-1, 1]^2$: Task A is a circle ($r^2 < 0.45$), Task B the half-plane $x_1 > x_2$.

Networks. Six networks, each with two hidden layers and tanh-type activations:

Network	Weights	Task readout	Parameters
Quaternion, S^3	$A \cdot q$, unit q on S^3, Riemannian updates	one output quaternion: A reads i, B reads j	292
Quaternion, free	$w \in \mathbb{R}^4$, Euclidean updates; same initial function	shared, as above	244
Quaternion, S^3, split	as S^3	two output quaternions, one per task	326
Quaternion, free, split	as free	split	272
Real MLP 14	2→14→14→2	a row of output weights per task	282
Real MLP 24	2→24→24→2, width-matched to the quaternion nets	per task	722

Protocol. Each seed trains a Task A checkpoint to at least 95% accuracy on a fixed 50×50 grid, then runs 1,000 steps of Task B (batch 20). The main metric is Task A accuracy at the moment Task B first reaches 95%. It compares networks at equal progress on the new task and barely moves with learning rate (66–67% across a 4× range for the S^3 network).

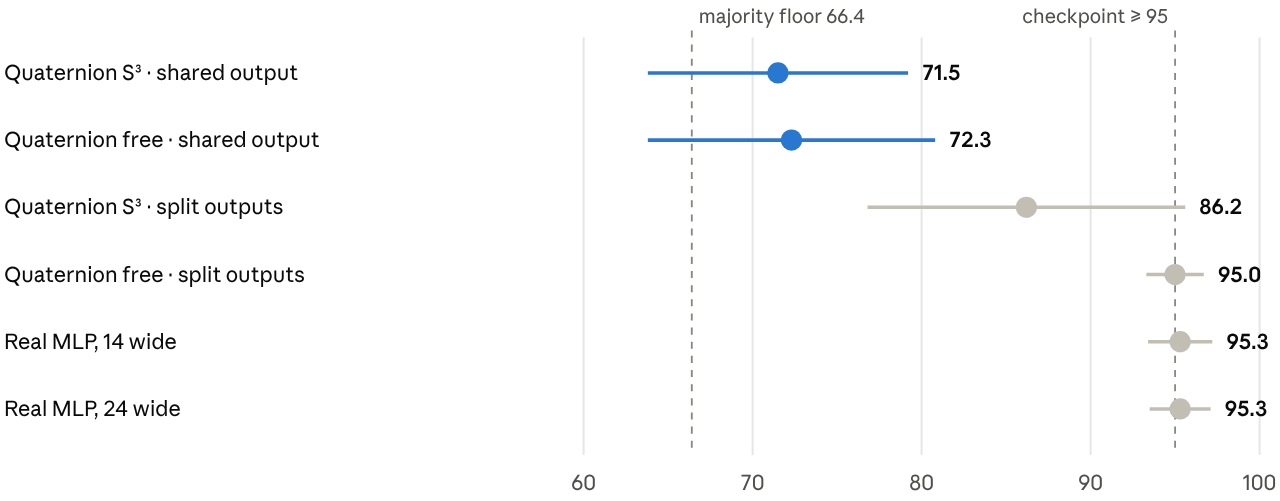
Seeds. Settings were tuned on seeds 1–6 and reported on seeds 7–12. Headline comparisons were then rerun on 24 fresh seeds (13–36). Intervals are 95% bootstrap intervals on the difference in means.

Result 1: the readout matters most

Giving each task its own output quaternion is worth 15–23 points of Task A. With that and free weights, the quaternion network forgets no less than a real MLP.

A shared output costs 15–23 points; with split outputs, quaternions match real MLPs

Task A accuracy (%) when Task B first reaches 95% · plain SGD · mean ± 1 sd, 24 fresh seeds



Blue: both tasks read one output quaternion. Grey: each task has its own output weights.

confirm.js · seeds 13–36 · lr 0.035

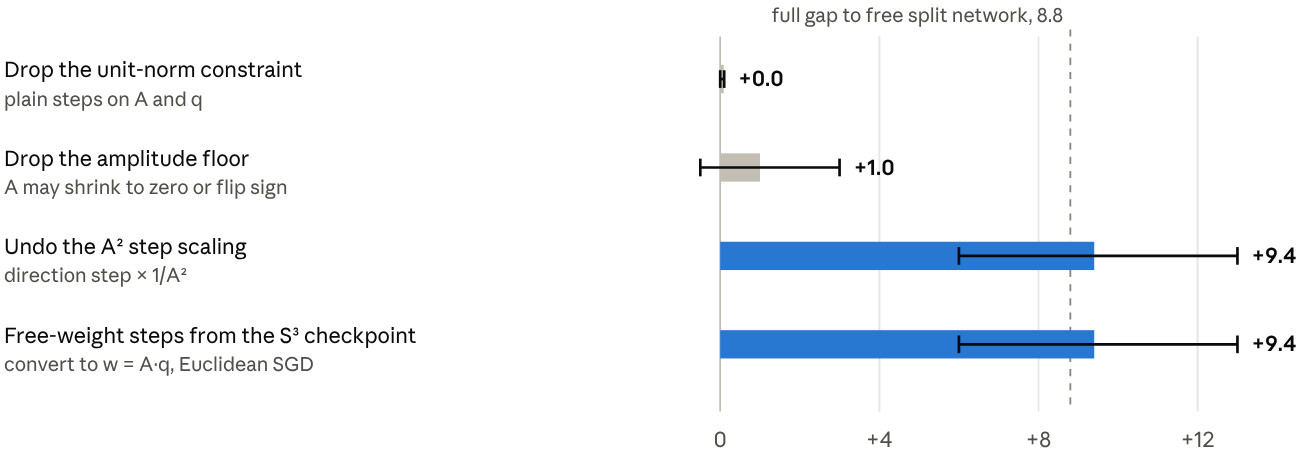
Splitting the output gains +14.7 points on S³ [9.5, 19.2] and +22.7 with free weights [19.0, 26.0]. With both fixes, the 272-parameter quaternion network is within 0.3 points of the 282- and 722-parameter MLPs (−0.3 [−1.2, 0.7] against either). The S³ split network still trails by 8.8 points [5.1, 12.7]; Result 2 traces that gap.

Result 2: the S³ cost is a step-size effect

The 8.8-point cost of S³ weights with split outputs comes from writing each weight as A·q, not from the sphere. Each variant below starts from the same S³ checkpoint and sees the same Task B batches; only the update differs.

The S³ cost is the A·q step scaling, not the sphere

Change in Task A accuracy (points) when Task B first reaches 95%, vs the S³ split network
Same checkpoint and batches per seed · paired over 24 fresh seeds · whiskers: 95% CI



Rescaling the S³ step and switching to free-weight steps agree within 0.25 points on every seed.

ablate.js · seeds 13–36 · lr 0.035 (lr 0.0175 gives +0.0, +0.8, +9.0, +9.0)

For w = A·q, the induced change in w is A·Δq + q·ΔA. Gradient descent on q sees the gradient scaled by A and moves w by a further factor of A, so direction changes shrink by A². The median A at the checkpoint is 0.18, so a typical direction step is about 30 times smaller than the free network's. Magnitude changes are unscaled. Task B is therefore learned mostly through amplitudes. It is learned more slowly too: 95% at step 80, against 35 once the scaling is undone.

The fix is a one-line preconditioner: divide the q step by A². That recovers the full gap and agrees with free-weight SGD to within 0.25 points on every seed. Where the S³ checkpoint landed is not the issue either: free-weight steps from it reach 95.6%, against 95.0% for the free network trained from scratch.

Under Adam the cost remains. On the same split-head checkpoint, Adam on free weights beats Adam on (A, q) by +9.4 to +10.6 points across three learning rates (0.0025–0.01; 95% CIs within [6.4, 14.2]). Adam normalizes the step on q, but the induced change in w is still scaled by A once. With a shared output there is no S³ effect under Adam either (−0.9 to 0.0). Adam forgets more than SGD on every network: at equal progress on Task B, 3–10 points more of Task A is lost.

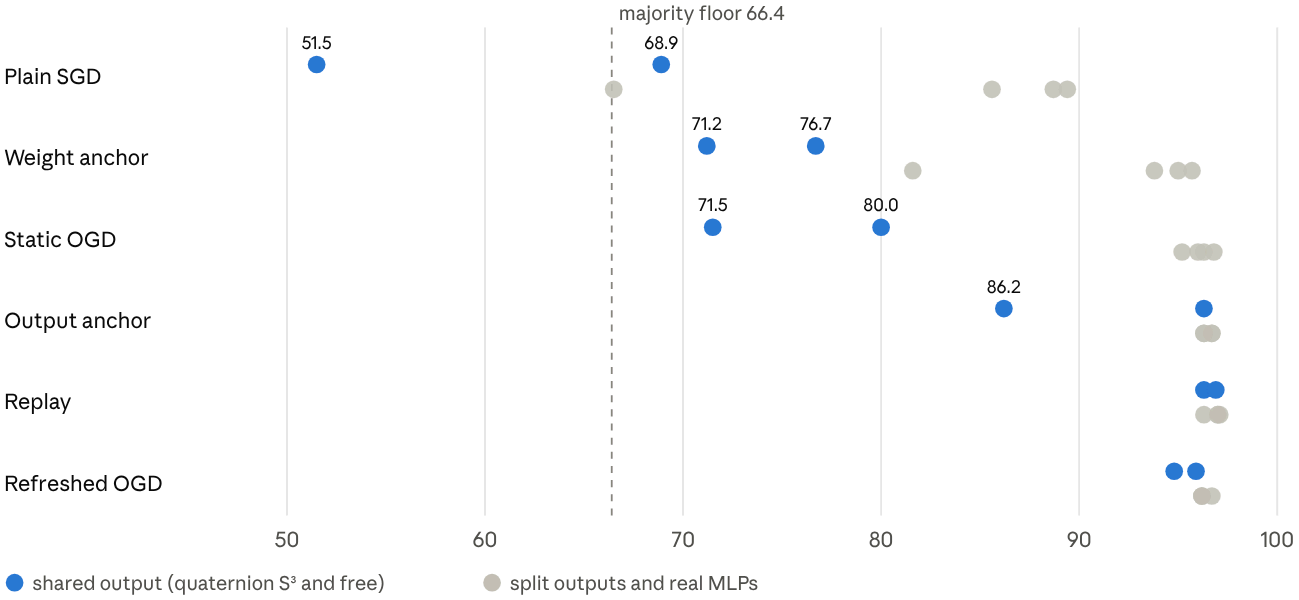
The length–direction split is the one used by weight normalization (Salimans and Kingma, 2016), whose gradients carry the same length scaling. The mechanism is not new; what this shows is its cost in forgetting.

Result 3: good protection makes the network irrelevant

With replay or refreshed OGD, every network keeps 95–97% of Task A, so architecture choice matters only when protection is weak.

Replay and refreshed OGD keep 95–97% everywhere; static OGD fails only with a shared output

Task A accuracy (%) at step 1,000, each method's tuned settings · held-out seeds 7–12 · one dot per network



archsweep.js · settings chosen on seeds 1–6, reported on seeds 7–12 · 100-point memory

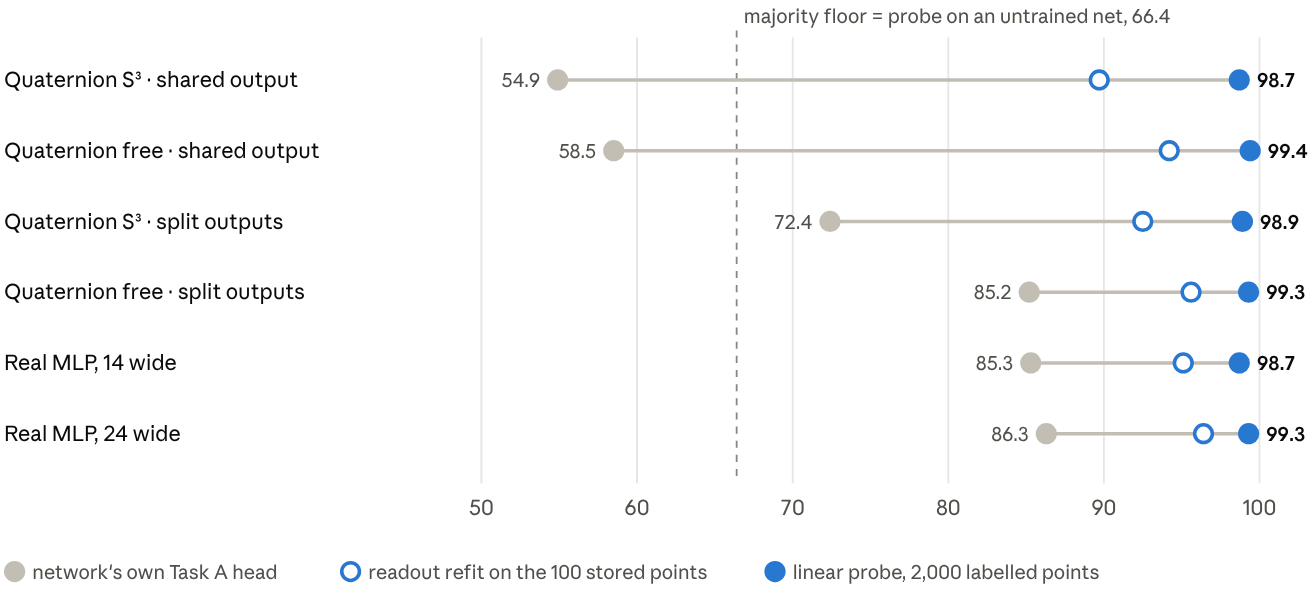
Output anchoring matches them on five of six networks; on the free shared-output network its tuned setting was unstable (86.2%, with Task B at 82.4%). Static OGD keeps 95–97% wherever the outputs are split, but only 72–80% with a shared output. Its failure, which looked like a limit of first-order protection, is mostly the shared readout. The weight anchor trails throughout and is weakest on the S³ networks.

Result 4: Task A is misread, not erased

After Task B, a linear probe on the last hidden layer reads Task A at 98.7–99.4% in every network. The lowest of 144 runs is 95.9%. The forgetting is in how the output reads the features, not in what the features hold.

After Task B, Task A is still readable from every network's features

Task A accuracy (%) after 1,000 SGD steps of Task B · mean of 24 fresh seeds (13–36)



probe.js · seeds 13–36 · lr 0.035 · probe fit by exact logistic regression, scored on the 50×50 grid

The probe is not finding the circle in arbitrary features: on the same networks untrained, it scores 66.4%, the majority floor. On split-output and real networks, Task B never updates Task A's output weights, so the frozen original head scores exactly what the live one does. Their 11–27-point loss is entirely the hidden code moving out from under a fixed readout.

Refitting only Task A's 25-parameter linear readout on the 100 stored points, after Task B, recovers 92.5–96.4% on the split-output and real networks. That is close to tuned replay (96–97%, measured on seeds 7–12), with no change to how Task B is trained. It is weakest on the S³ shared-output network (89.7%, as low as 68.8% on one seed). This replicates, in a toy setting, the gap between accuracy forgetting and probe forgetting reported by Davari et al. (CVPR 2022).

Related work

No peer-reviewed study of forgetting in quaternion networks turned up. Results 1 and 4 replicate known patterns, and Result 2's mechanism is known but its cost in forgetting is not reported.

Result	Closest prior work	How this relates
1. Shared readout	van de Ven et al., 2022; Heads Collapse, Features Stay, ICLR 2026	Known: shared output heads forget far more than per-task heads
2. A^2 step scaling	Salimans and Kingma, 2016; Lyle et al., 2024; Hägele et al., 2026	Mechanism known; its effect on plasticity and training speed is studied, not on forgetting
4. Misread, not erased	Davari et al., 2022; Ramasesh et al., 2020; Heads Collapse, ICLR 2026	Replication in a quaternion setting
Quaternion continual learning	Quaternion Phase Multiplexing, preprint draft, Sep 2026	Unit-quaternion trunk with per-task heads; untested against the split-readout control

The Quaternion Phase Multiplexing draft reports 94.6% retention from one seed on synthetic tasks, with per-task decision heads and no standard baselines. Result 1 suggests those heads, not the quaternion phases, could carry most of that retention.

What did not replicate, and limits

One exploratory result failed on fresh seeds. On seeds 1–12, the free split-head quaternion network kept 2.6 points more of Task A than the parameter-matched real MLP [0.8, 4.8]. On seeds 13–36 the difference was −0.3 [−1.2, 0.7]. The S^3 effect with a shared readout was also unstable: +12 on the first seeds, +0.8 [−4.2, 5.3] on the fresh ones.

The limits are those of a toy. There is one task pair, two-dimensional inputs, networks of a few hundred parameters, and SGD or Adam for Task B. Adam rescales the A^2 effect but does not remove it (Result 2); momentum SGD is untested. Why amplitude-heavy updates disturb Task A more is not tested here.

Three things would make this worth more: the 2×2 and the A^2 ablation on a standard benchmark such as split MNIST with quaternion layers; the A^2 cost under the magnitude–direction decoupled optimizers of Hägele et al.; and a Google Scholar pass on quaternion and hypercomplex incremental learning, which this search did not cover.

Reproducing

Every number comes from a seeded JavaScript engine with no dependencies, checked against finite-difference gradients (relative error at most 3×10^{-6}). The companion page, `qcl_workbench.html`, runs the same engine in the browser. A live run or a multi-seed benchmark on any of the six networks reproduces the study data exactly.

- Checkpoint RNG: `seed·7919+1`; memory: `seed·7919+2`; Task B stream: `seed·104729+3` (mulberry32).
- Study: `archsweep.js` (seeds 1–12), `confirm.js` (seeds 13–36), `ablate.js` (the S³-cost ablations), `adam.js` (Task B with Adam), `probe.js` (linear probes and readout refits).